

# Lightweight Statistics & Monitoring Instrumentation

LLDAP · Dex · OpenFGA · Vault · libcloud REST · Cloud Providers

## Overview

This document catalogs the specific metrics, statistics, and instrumentation flags for every component in the stack, organised by collection layer (host, Docker, process), and maps them to performance, security, and throughput dashboards. The goal is the minimal high-signal set — not every available metric, only those with actionable meaning.

All process-level metrics converge into Prometheus → Grafana. Host and Docker metrics come from `node_exporter` (:9100) and `cAdvisor` (:8080). Traces flow via the `OpenTelemetry Collector` to Jaeger or Tempo.<sup>[1][2][^3]</sup>

## Layer 0 — Host & Docker (Universal, Applies to All Components)

These run as sidecars and require zero instrumentation changes to the application containers.

### `node_exporter` (host-level, port 9100)<sup>[4][1]</sup>

| Metric                                             | Type    | Dashboard Panel                                  |
|----------------------------------------------------|---------|--------------------------------------------------|
| <code>node_cpu_seconds_total{mode="idle"}</code>   | Counter | CPU utilisation % per core                       |
| <code>node_cpu_seconds_total{mode="iowait"}</code> | Counter | I/O wait — signals storage bottleneck            |
| <code>node_memory_MemAvailable_bytes</code>        | Gauge   | Available RAM                                    |
| <code>node_memory_SwapFree_bytes</code>            | Gauge   | Swap usage — signals memory pressure             |
| <code>node_filesystem_avail_bytes</code>           | Gauge   | Disk free — alert on audit log partition         |
| <code>node_disk_io_time_seconds_total</code>       | Counter | Disk saturation (Vault audit device, PostgreSQL) |
| <code>node_network_receive_bytes_total</code>      | Counter | Inbound throughput                               |
| <code>node_network_transmit_bytes_total</code>     | Counter | Outbound throughput                              |
| <code>node_load1 / node_load15</code>              | Gauge   | Load average                                     |

**Key alert:** `node_filesystem_avail_bytes` on the Vault audit log partition — Vault blocks all requests if it cannot write audit logs.<sup>[^5]</sup>

cAdvisor (container-level, port 8080)<sup>[6][2]</sup>

| Metric                                            | Type    | Dashboard Panel                                 |
|---------------------------------------------------|---------|-------------------------------------------------|
| container_cpu_usage_seconds_total{name="tainer>"} | Counter | Per-container CPU %                             |
| container_memory_usage_bytes{name="tainer>"}      | Gauge   | Per-container RAM                               |
| container_memory_working_set_bytes                | Gauge   | Actual working set (excludes reclaimable cache) |
| container_network_receive_bytes_total             | Counter | Per-container inbound traffic                   |
| container_network_transmit_bytes_total            | Counter | Per-container outbound traffic                  |
| container_fs_reads_bytes_total                    | Counter | Container disk read throughput                  |
| container_fs_writes_bytes_total                   | Counter | Container disk write throughput                 |
| container_last_seen                               | Gauge   | Container alive check                           |
| container_oom_events_total                        | Counter | OOM kills — critical alert                      |
| container_restart_count                           | Counter | Restart count — detect crash loops              |

Recommended Grafana dashboard IDs: Node Exporter Full (1860), Docker cAdvisor (21154).<sup>[6][4]</sup>

Layer 1 — Dex (OIDC Provider)

Enabling Metrics

Dex exposes Prometheus metrics natively. Add to config.yaml:

```
telemetry:
  http: "0.0.0.0:5558"
```

Scrape target: dex:5558/metrics

Instrumentation Flags

| Flag / Config                    | Purpose                                                                 |
|----------------------------------|-------------------------------------------------------------------------|
| logger.level: debug              | Log individual LDAP bind attempts, token issuance, and connector events |
| logger.format: json              | Structured log output for SIEM ingestion                                |
| oauth2.skipApprovalScreen: false | Log consent screen interactions for audit                               |

## Key Metrics

| Metric                                          | Type      | Dashboard Panel                               |
|-------------------------------------------------|-----------|-----------------------------------------------|
| dex_kubernetes_request_total                    | Counter   | Total Kubernetes API requests (if applicable) |
| http_requests_total{handler="/token"}           | Counter   | Token issuance rate                           |
| http_requests_total{handler="/auth"}            | Counter   | Authorization request rate                    |
| http_request_duration_seconds{handler="/token"} | Histogram | Token endpoint latency (P50, P95, P99)        |
| dex_connector_ldap_bind_total                   | Counter   | LDAP bind attempts against LLDAP/AD           |
| dex_connector_ldap_bind_errors_total            | Counter   | LDAP bind failures — security signal          |
| http_requests_total{code="401"}                 | Counter   | Authentication failures                       |
| http_requests_total{code="500"}                 | Counter   | Internal errors                               |
| go_goroutines                                   | Gauge     | Runtime goroutine count                       |
| go_memstats_heap_inuse_bytes                    | Gauge     | Heap memory pressure                          |
| process_open_fds                                | Gauge     | Open file descriptors                         |

## Security-Significant Log Events to Parse

```
{ "level": "error", "msg": "Failed to authenticate", "username": "alice", "remote": "10.0.0.1" }
{ "level": "info", "msg": "login successful", "connector": "ldap", "username": "alice" }
{ "level": "info", "msg": "keys rotated" }
```

**Alert on:** dex\_connector\_ldap\_bind\_errors\_total rate spike → brute-force attempt; keys rotated outside scheduled rotation window → signing key tampered.

## Layer 2 — LLDAP

### Enabling Metrics & Verbose Output

LLDAP does not natively expose Prometheus metrics. Use two approaches:

**Option A — structured JSON logging** (environment variable):

```
RUST_LOG=lldap=debug
```

This emits per-operation JSON logs: bind, search, add, modify, delete.

**Option B — LDAP access log exporter:** Use ldap\_exporter or parse LLDAP access logs with the Prometheus mtail or grok\_exporter sidecar to derive counters.

**Option C — PostgreSQL backend metrics:** If LLDAP uses Postgres as its backend, Postgres query metrics via postgres\_exporter give indirect throughput data.

**Derived Metrics (from log parsing)**

| Derived Metric                                     | Source    | Dashboard Panel                                |
|----------------------------------------------------|-----------|------------------------------------------------|
| lldap_bind_total{result="success\ fail"}           | Log parse | Authentication rate and failure rate           |
| lldap_search_total{base="..."}                     | Log parse | Directory query throughput                     |
| lldap_modify_total{op="add_member\ remove_member"} | Log parse | Group membership change rate — security signal |
| lldap_user_create_total                            | Log parse | Account provisioning rate                      |
| lldap_user_delete_total                            | Log parse | Account deletion rate                          |
| lldap_bind_duration_ms                             | Log parse | Bind latency histogram                         |

**Security-Significant Log Events**

```
BIND dn="uid=alice,ou=people,dc=example,dc=com" result=INVALID_CREDENTIALS src=1.2.3
MODIFY dn="cn=cloud-owners,ou=groups" add:member val="uid=eve"
```

**Alert on:** >5 consecutive INVALID\_CREDENTIALS from same source IP; add:member where target group contains owner or admin in name.

**Layer 3 — OpenFGA**

**Enabling Metrics & Tracing**

OpenFGA exposes Prometheus metrics by default on :2112/metrics.[^7]

```
# Enable RPC latency histograms (disabled by default, adds ~5% overhead)
OPENFGA_METRICS_ENABLE_RPC_HISTOGRAMS=true

# Enable distributed tracing (OTLP export)
OPENFGA_TRACE_ENABLED=true
OPENFGA_TRACE_OTLP_ENDPOINT=otel-collector:4317
OPENFGA_TRACE_SAMPLE_RATIO=0.1 # 10% sampling for production

# Structured JSON logging
OPENFGA_LOG_FORMAT=json
OPENFGA_LOG_LEVEL=info # raise to debug for troubleshooting only
```

```
# Enable pprof profiler (only when investigating performance issues)
# openfga run --profiler-enabled --profiler-addr :3001
```

Key Metrics<sup>[8]</sup>[3][^7]

| Metric                                                         | Type      | Dashboard Panel                             |
|----------------------------------------------------------------|-----------|---------------------------------------------|
| grpc_server_handled_total{grpc_method="Check"}                 | Counter   | Authorization check throughput              |
| grpc_server_handled_total{grpc_method="Write"}                 | Counter   | Tuple write rate                            |
| grpc_server_handled_total{grpc_method="Read"}                  | Counter   | Tuple read rate                             |
| grpc_server_handled_total{grpc_code="OK"}                      | Counter   | Successful requests                         |
| grpc_server_handled_total{grpc_code!="OK"}                     | Counter   | Error rate                                  |
| grpc_server_handling_seconds_bucket{grpc_method="Check"}       | Histogram | Check latency P50/P95/P99                   |
| grpc_server_handling_seconds_bucket{grpc_method="ListObjects"} | Histogram | ListObjects latency (expensive op)          |
| openfga_datastore_query_count                                  | Counter   | Datastore queries per authorization check   |
| openfga_request_duration_by_query_count_ms                     | Histogram | Correlation of check latency vs graph depth |
| go_goroutines                                                  | Gauge     | Runtime health                              |
| go_memstats_heap_inuse_bytes                                   | Gauge     | Memory pressure                             |
| process_cpu_seconds_total                                      | Counter   | CPU consumption                             |

Security-Significant Log Fields to Monitor

```
{ "grpc_method": "Check", "allowed": false, "user": "user:alice", "object": "api:com
{ "grpc_method": "Write", "store_id": "...", "client_ip": "10.0.1.99" }
```

**Alert on:** `grpc_server_handled_total{grpc_method="Check", grpc_code!="OK"}` spike → model or auth error; high rate of `allowed=false` for a single user → permission probe; Write calls from IPs outside the reconciler CIDR → unauthorised tuple manipulation.

## Health Endpoint

```
curl https://openfga:8080/healthz
# {"status":"SERVING"}
```

## Layer 4 — HashiCorp Vault

### Enabling Telemetry<sup>[9]</sup><sup>[10]</sup>

Add to `config.hcl`:

```
telemetry {
  prometheus_retention_time = "30s"
  disable_hostname          = false
  enable_hostname_label     = true
  filter_default            = false
  prefix_filter = [
    "+vault.core",
    "+vault.token",
    "+vault.expire",
    "+vault.audit",
    "+vault.runtime",
    "+vault.auth.ldap"
  ]
}
```

Scrape target: `vault:8200/v1/sys/metrics?format=prometheus` (requires valid Vault token with `sys/metrics` policy).<sup>[11]</sup>

Also enable at least two audit devices:<sup>[15]</sup>

```
vault audit enable file file_path=/var/log/vault/audit.log
vault audit enable syslog
```

### Key Metrics<sup>[12]</sup><sup>[13]</sup><sup>[14]</sup>

#### Core / Health

| Metric                                                 | Type    | Alert Threshold                 |
|--------------------------------------------------------|---------|---------------------------------|
| <code>vault.core.handle_request</code> (summary)       | Summary | P99 > 500ms → investigate       |
| <code>vault.core.handle_login_request</code> (summary) | Summary | P99 > 1s → scale or investigate |

| Metric                                          | Type    | Alert Threshold                           |
|-------------------------------------------------|---------|-------------------------------------------|
| <code>vault.core.leadership_lost</code>         | Summary | Any occurrence → HA instability           |
| <code>vault.core.leadership_setup_failed</code> | Counter | Any occurrence → cluster fault            |
| <code>vault.core.post_unseal</code>             | Summary | Spike → audit device or plugin issue      |
| <code>vault.core.unsealed</code>                | Gauge   | 0 → Vault is sealed, all requests blocked |

## Token & Lease

| Metric                                                | Type    | Alert Threshold                            |
|-------------------------------------------------------|---------|--------------------------------------------|
| <code>vault.token.creation{auth_method="ldap"}</code> | Counter | Baseline + $3\sigma$ → unusual auth volume |
| <code>vault.expire.num_leases</code>                  | Gauge   | Sustained growth → lease accumulation risk |
| <code>vault.expire.revoke (summary)</code>            | Summary | P99 > 2s → revocation backlog              |
| <code>vault.expire.renew-token (summary)</code>       | Summary | P99 > 500ms → renewal latency              |
| <code>vault.expire.fetch-lease-times</code>           | Summary | Latency indicator for lease lookup         |

## Auth

| Metric                                       | Type    | Dashboard Panel                     |
|----------------------------------------------|---------|-------------------------------------|
| <code>vault.auth.ldap.login_request</code>   | Summary | LDAP auth latency                   |
| <code>vault.auth.ldap.login (counter)</code> | Counter | Successful LDAP logins per interval |

## Storage Backend

| Metric                                    | Type    | Alert Threshold                        |
|-------------------------------------------|---------|----------------------------------------|
| <code>vault.&lt;storage&gt;.get</code>    | Summary | P99 > 50ms → storage degradation       |
| <code>vault.&lt;storage&gt;.put</code>    | Summary | P99 > 100ms → storage write bottleneck |
| <code>vault.&lt;storage&gt;.list</code>   | Summary | Baseline + $3\sigma$                   |
| <code>vault.&lt;storage&gt;.delete</code> | Summary | Baseline + $3\sigma$                   |

## Audit

| Metric                                        | Type    | Alert Threshold                                   |
|-----------------------------------------------|---------|---------------------------------------------------|
| <code>vault.audit.log_request_failure</code>  | Counter | Any non-zero → <b>critical</b> , Vault will block |
| <code>vault.audit.log_response_failure</code> | Counter | Any non-zero → <b>critical</b>                    |
| <code>vault.audit.log.request</code>          | Summary | P99 > 100ms → audit device I/O issue              |

Runtime

| Metric                                    | Type    | Alert Threshold                      |
|-------------------------------------------|---------|--------------------------------------|
| <code>vault.runtime.num_goroutines</code> | Gauge   | Spike > 2x baseline                  |
| <code>vault.runtime.heap_objects</code>   | Gauge   | Sustained growth → memory leak       |
| <code>vault.runtime.gc_pause_ns</code>    | Summary | > 5 billion ns/min → memory pressure |
| <code>vault.runtime.sys_bytes</code>      | Gauge   | > 90% host RAM → add memory          |

Security-Only Metrics (derived from audit log parsing)

| Derived Metric                                                   | Trigger                                              |
|------------------------------------------------------------------|------------------------------------------------------|
| <code>vault_root_token_usage_total</code>                        | Any <code>auth.display_name=root</code> in audit log |
| <code>vault_policy_modification_total</code>                     | Write to <code>sys/policies/acl/*</code>             |
| <code>vault_audit_device_change_total</code>                     | Write/delete to <code>sys/audit/*</code>             |
| <code>vault_credential_request_rate{path="aws/creds/..."}</code> | Count per minute per requester                       |

Layer 5 — libcloud REST API

libcloud REST is a custom service and must be instrumented by the developer. The following is the recommended minimum instrumentation using the Python `prometheus_client` library (if Python/FastAPI) or equivalent in Go.

Instrumentation to Add<sup>[15]</sup>

```
from prometheus_client import Counter, Histogram, Gauge

# Counters
http_requests_total = Counter(
    'libcloud_http_requests_total',
    'Total HTTP requests',
    ['method', 'path', 'provider', 'status_code']
)
openfga_check_total = Counter(
    'libcloud_openfga_check_total',
    'OpenFGA authorization checks',
    ['result']          # allowed / denied
)
vault_credential_fetch_total = Counter(
    'libcloud_vault_credential_fetch_total',
    'Vault dynamic credential requests',
    ['provider', 'role', 'status']
)
cloud_api_call_total = Counter(
    'libcloud_cloud_api_call_total',
    'Outbound cloud provider API calls',
```

```

    ['provider', 'operation', 'status']
)
auth_failure_total = Counter(
    'libcloud_auth_failure_total',
    'JWT validation failures',
    ['reason']          # expired / invalid_sig / bad_aud / bad_iss
)

# Histograms
request_duration = Histogram(
    'libcloud_request_duration_seconds',
    'End-to-end request latency',
    ['method', 'path', 'provider']
)
openfga_check_duration = Histogram(
    'libcloud_openfga_check_duration_seconds',
    'Time spent waiting for OpenFGA Check response'
)
vault_fetch_duration = Histogram(
    'libcloud_vault_fetch_duration_seconds',
    'Time to fetch dynamic credential from Vault'
)
cloud_api_duration = Histogram(
    'libcloud_cloud_api_duration_seconds',
    'Cloud provider API call latency',
    ['provider', 'operation']
)

# Gauges
active_requests = Gauge(
    'libcloud_active_requests',
    'Currently in-flight requests'
)

```

### Key Metrics Dashboard

| Metric                                                    | Dashboard Panel                             |
|-----------------------------------------------------------|---------------------------------------------|
| libcloud_http_requests_total by status_code               | HTTP traffic volume and error rate          |
| libcloud_request_duration_seconds{quantile="0.99"}        | End-to-end P99 latency                      |
| libcloud_openfga_check_total{result="denied"} rate        | Authorization denial rate — security signal |
| libcloud_openfga_check_duration_seconds{quantile="0.95"}  | OpenFGA latency contribution                |
| libcloud_vault_credential_fetch_total{status="error"}     | Vault availability issues                   |
| libcloud_vault_fetch_duration_seconds{quantile="0.95"}    | Vault latency contribution                  |
| libcloud_cloud_api_call_total{status="error"} by provider | Cloud provider error rate                   |
| libcloud_cloud_api_duration_seconds by provider+operation | Cloud API latency per provider              |
| libcloud_auth_failure_total by reason                     | JWT attack surface visibility               |

| Metric                   | Dashboard Panel                        |
|--------------------------|----------------------------------------|
| libcloud_active_requests | Concurrency / connection pool pressure |

Structured Access Log Fields (every request)

```
{
  "ts": "2026-07-04T10:00:00Z",
  "request_id": "uuid-v4",
  "traceparent": "00-trace-span-01",
  "user": "alice",
  "source_ip": "10.0.1.5",
  "method": "POST",
  "path": "/compute/aws/nodes",
  "provider": "aws",
  "fga_result": "allowed",
  "fga_duration_ms": 4,
  "vault_lease_id": "aws/creds/ec2-admin/xyz",
  "vault_duration_ms": 18,
  "cloud_request_id": "req-0a1b2c3d",
  "cloud_duration_ms": 312,
  "status_code": 201,
  "total_duration_ms": 341
}
```

Layer 6 — Apache libcloud (Library Layer)

Apache libcloud itself is a library with no built-in metrics endpoint. Instrument at the wrapper level within libcloud REST:

| What to Instrument               | How                                                                                                                                      |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Cloud driver initialisation time | Timer around <code>get_driver()</code> + <code>connect()</code>                                                                          |
| Per-provider API call latency    | Timer wrapping every libcloud driver method call                                                                                         |
| Connection pool state            | Log when a new connection to provider is opened vs reused                                                                                |
| Retry count                      | Counter increment on each libcloud retry attempt                                                                                         |
| Exception types                  | Counter by exception class (e.g. <code>InvalidCredsException</code> , <code>RateLimitExceededError</code> , <code>LibcloudError</code> ) |

Enable libcloud's built-in debug logging for verbose HTTP-level output (use in development / troubleshooting only — too verbose for production):

```
import logging
logging.getLogger('libcloud.common.base').setLevel(logging.DEBUG)
```

```
# or via env:
# LIBCLOUD_DEBUG=/var/log/libcloud_http.log
```

LIBCLOUD\_DEBUG writes full HTTP request/response bodies to a file — useful for diagnosing provider API errors but must **never** be enabled in production as it logs credentials in plaintext. [^16]

## Layer 7 — Cloud Provider Native Telemetry

These are pulled from cloud provider APIs or forwarded to the central Prometheus/SIEM, not scraped via node\_exporter.

| Provider | Source                             | Key Data Points                                                                                                                            |
|----------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| AWS      | CloudWatch + CloudTrail            | API call counts, IAM throttling (ThrottlingException), STS credential issuance rate, EC2/S3 operation counts, region distribution of calls |
| GCP      | Cloud Monitoring + Audit Logs      | API request counts by method, quota utilisation, service account key usage                                                                 |
| Azure    | Azure Monitor + Activity Log       | ARM API call rate, throttling (HTTP 429 rate), credential usage by client ID                                                               |
| Alibaba  | CloudMonitor + ActionTrail         | API call frequency, geographic source of calls, AccessKey usage                                                                            |
| Nutanix  | Prism API (/api/nutanix/v3/events) | VM create/delete events, storage pool utilisation, host health                                                                             |

## Minimum Cloud-Side Metrics to Pull

| Metric                                                       | Security / Ops Value                                        |
|--------------------------------------------------------------|-------------------------------------------------------------|
| API call rate per credential (AccessKey / service principal) | Detect credential misuse — spikes after hours are anomalous |
| HTTP 429 (throttle) rate                                     | Operational — libcloud retry storms                         |
| Geographic source of API calls                               | Security — calls from unexpected regions                    |
| Failed API calls by error code                               | AuthFailure, AccessDenied → stolen/expired credential       |
| Resource creation/deletion rate                              | Ops — sudden destroy spike is critical alert                |

## Unified Prometheus Scrape Configuration

```
scrape_configs:
  - job_name: node-exporter
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: cadvisor
    static_configs:
      - targets: ['cadvisor:8080']
```

```

- job_name: dex
  static_configs:
    - targets: ['dex:5558']

- job_name: openfga
  static_configs:
    - targets: ['openfga:2112']

- job_name: vault
  metrics_path: /v1/sys/metrics
  params:
    format: ['prometheus']
  bearer_token: '<vault-metrics-token>'
  static_configs:
    - targets: ['vault:8200']

- job_name: libcloud-rest
  static_configs:
    - targets: ['libcloud-rest:9091']    # expose /metrics on dedicated port

```

## Recommended Grafana Dashboard Layout

| Dashboard                                | Panels                                                                                                                                                        |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Host &amp; Container Health</b>       | CPU %, RAM %, Disk %, Network in/out, Container restarts, OOM events                                                                                          |
| <b>Identity &amp; Auth (Dex + LLDAP)</b> | Token issuance rate, LDAP bind rate, LDAP bind error rate, bind latency P99, auth failure by user                                                             |
| <b>Authorization (OpenFGA)</b>           | Check throughput, Check latency P99, Allow vs Deny ratio, Write/Delete tuple rate, Datastore query count                                                      |
| <b>Secrets &amp; Vault</b>               | Token creation rate, Active leases, Revoke latency, Storage backend latency, Audit device failures, Root token usage, Sealed/unsealed gauge                   |
| <b>libcloud REST API</b>                 | Request rate, P99 latency, FGA denial rate, Vault fetch latency, Cloud API error rate by provider, JWT failure by reason                                      |
| <b>Cloud Providers</b>                   | API call rate by provider/credential, Throttle rate (429), Geographic anomaly, Destroy event rate                                                             |
| <b>Security Signals</b>                  | LLDAP group modification rate, OpenFGA unexpected Write source IPs, Vault root token usage, Cloud API calls with no libcloud match, Auth failure by source IP |

## Instrumentation Overhead Summary

| Component     | Metrics Overhead     | Logging Overhead | Notes     |
|---------------|----------------------|------------------|-----------|
| node_exporter | <0.1% CPU, <30MB RAM | Minimal          | Always on |
| cAdvisor      | ~0.5% CPU, ~50MB RAM | Minimal          | Always on |
| Dex           | Negligible           | Low (JSON info)  | Always on |

| Component      | Metrics Overhead                                 | Logging Overhead                   | Notes                            |
|----------------|--------------------------------------------------|------------------------------------|----------------------------------|
| LLDAP          | Negligible (log parse)                           | Low-Medium (debug)                 | Use info in prod                 |
| OpenFGA        | <1% (without histograms) / ~5% (with histograms) | Low (JSON info)                    | Enable histograms in prod[^7]    |
| Vault          | <2% with audit + telemetry                       | Medium (two audit devices)         | Both audit devices mandatory[^5] |
| libcloud REST  | ~1% (Prometheus client)                          | Medium (structured JSON)           | Always on                        |
| LIBCLOUD_DEBUG | N/A                                              | <b>Very high — plaintext creds</b> | <b>Dev only</b>                  |

## References

1. [Monitoring Container and Node Metrics Using Prometheus, cAdvisor ...](#) - Prometheus is one of the best open source tools out there to monitor aggregated metrics related to s...
2. [Monitoring Docker container metrics using cAdvisor](#) - cAdvisor (short for container Advisor) analyzes and exposes resource usage and performance data from...
3. [Supported Attributes](#) - How to configure your SDK Client to collect telemetry using OpenTelemetry.
4. [Grafana, Prometheus, cAdvisor, Node Exporter & Docker](#) - If you're at an intermediate level and just starting to run your applications on a VPS or server, ev...
5. [Audit logging best practices | Vault - HashiCorp Developer](#) - Recommendations for setting up audit logging in HashiCorp Vault.
6. [Just do Grafana — Monitor Docker containers with cadvisor ...](#) - In this guide i will show you how i monitor my docker servers in Grafana with cadvisor and Node expor...
7. [Configuring OpenFGA](#) - Configuring an OpenFGA Server
8. [Running OpenFGA in Production](#) - The following list outlines best practices for running OpenFGA in a production environment. Cluster ...
9. [Enable Vault telemetry gathering](#) - Step-by-step guide to enabling telemetry gathering with Vault
10. [Monitor telemetry with Prometheus & Grafana | Vault](#) - Learn about monitoring Vault telemetry metrics with Grafana and Prometheus. A hands-on scenario is i...
11. [vault/website/content/api-docs/system/metrics.mdx at main · hashicorp/vault](#) - A tool for secrets management, encryption as a service, and privileged access management - hashicorp...
12. [CodeKloud Notes](#) - Comprehensive course notes and guides for cloud technologies, DevOps, Kubernetes, Docker, and more
13. [Key metrics for common health checks | Vault - HashiCorp Developer](#) - Vault creates a lease when it generates a dynamic secret or service token. This lease contains essen...
14. [Telemetry reference: Core system metrics | Vault](#) - Technical reference for core system telemetry values.

15. Prometheus Metrics - MCP Server with LangGraph - Set up Prometheus for metrics collection, custom business metrics, and application monitoring
16. Using Apache Libcloud for declarative and procedural multi ... - Apache Libcloud is a Python library which hides differences between different cloud provider APIs an...